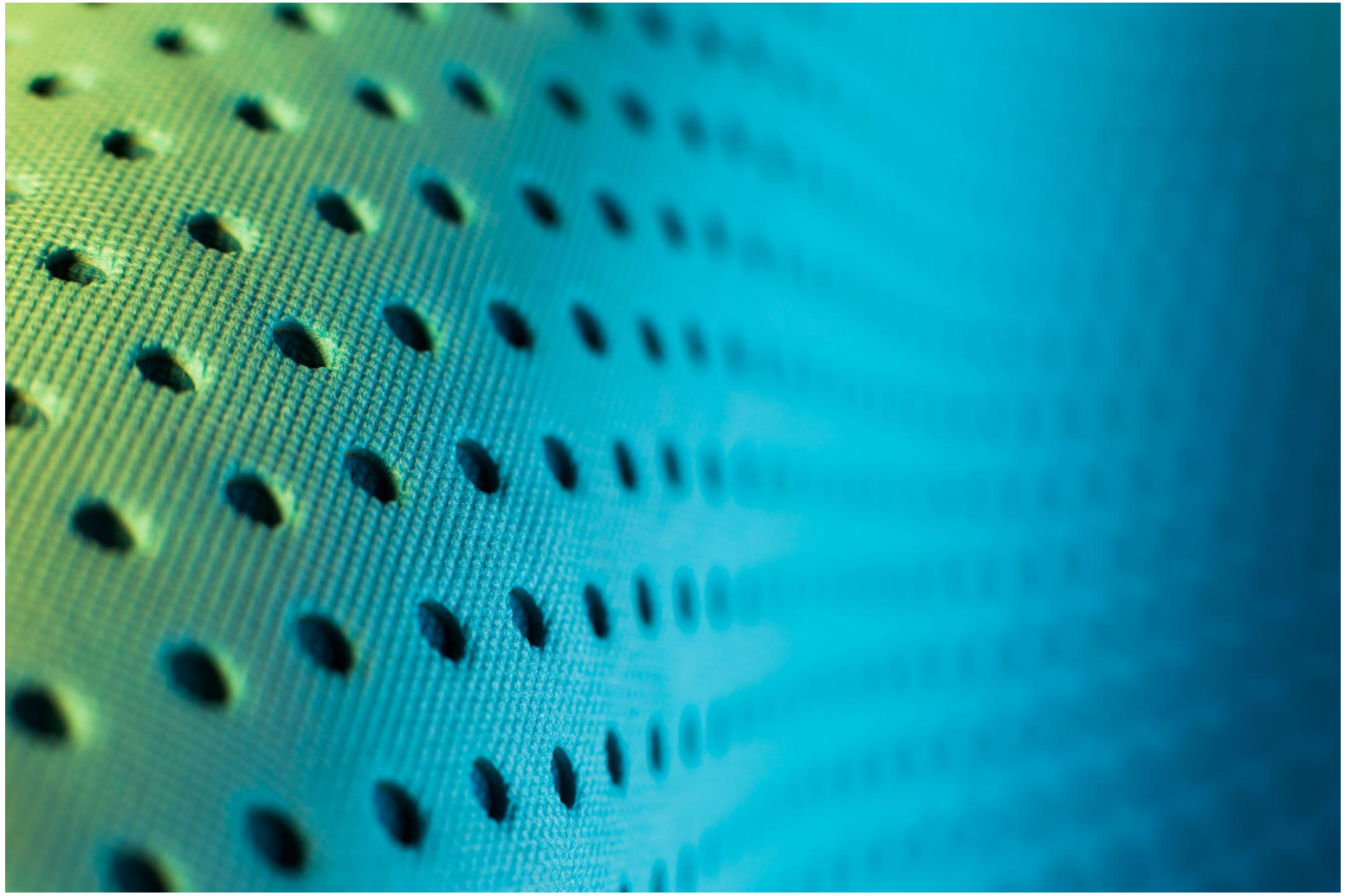
Introdução a Autenticação



Autenticação e Autorização

- Autenticação verifica a identidade de quem tenta acessar o sistema.
- Autorização determina quais recursos e operações um usuário autenticado pode acessar.
- Assim, um usuário pode estar corretamente autenticado e, ainda assim, não ter permissão para executar determinada operação.

Mecanismos de autenticação/autorização

- Os mecanismos de autenticação e autorização evoluíram ao longo do tempo.
- Inicialmente, essas verificações eram implementadas diretamente na lógica de negócio da aplicação.
- Com o crescimento das aplicações Web e das exigências de segurança, frameworks passaram a centralizar essas responsabilidades na fronteira da aplicação.

Mecanismo de autenticação básico

- HTTP Basic Authentication é um mecanismo simples para o cliente enviar suas credenciais ao acessar um recurso protegido.
- O cliente combina usuário e senha no formato usuário:senha, codifica esse valor em Base64 e o envia no cabeçalho Authorization.
- Base64 é apenas uma codificação, não criptografia. Por isso, HTTP Basic deve ser usado sobre HTTPS.

Detalhando a autenticação básica

Desafio

Quando o cliente solicita acesso a um recurso protegido o servidor responde com o código 401 e cabeçalho “WWW-Authenticate” indicando que autenticação é necessária..

Credenciais

Ao receber o desafio o cliente solicita o usuário e senha e os codifica usando o formato Base64 combinando os dois com um “:”

Cabecalho de autorização

O cliente inclui as credenciais codificadas no cabeçalho da requisição usando o formato:

Basic <encoded_credentials>

Verificação

O servidor recebe o cabeçalho de Autorização, decodifica a string “Base64”, extrai o usuário e a senha e verifica os mesmos contra o banco de dados de usuários.

Sucesso/Falha

Se as credenciais são válidas, o servidor permite o acesso ao recurso requisitado; caso contrário o servidor responde com outro erro 401 (unauthorized) ou 403 (proibido).

Observações

- Codificação Base64:
- Base64 não é criptografia. As credenciais podem ser facilmente recuperadas a partir do valor codificado.
- Requisito: HTTPS
- HTTPS protege as credenciais durante o transporte entre cliente e servidor.
- Considerações de segurança:
- HTTP Basic é simples e útil em determinados cenários, mas oferece menos recursos que mecanismos baseados em tokens.
- Alternativas:
- OAuth 2.0 e mecanismos baseados em tokens, como JWT, são comuns quando são necessários recursos adicionais de autenticação e autorização.

401 × 403: autenticação e autorização

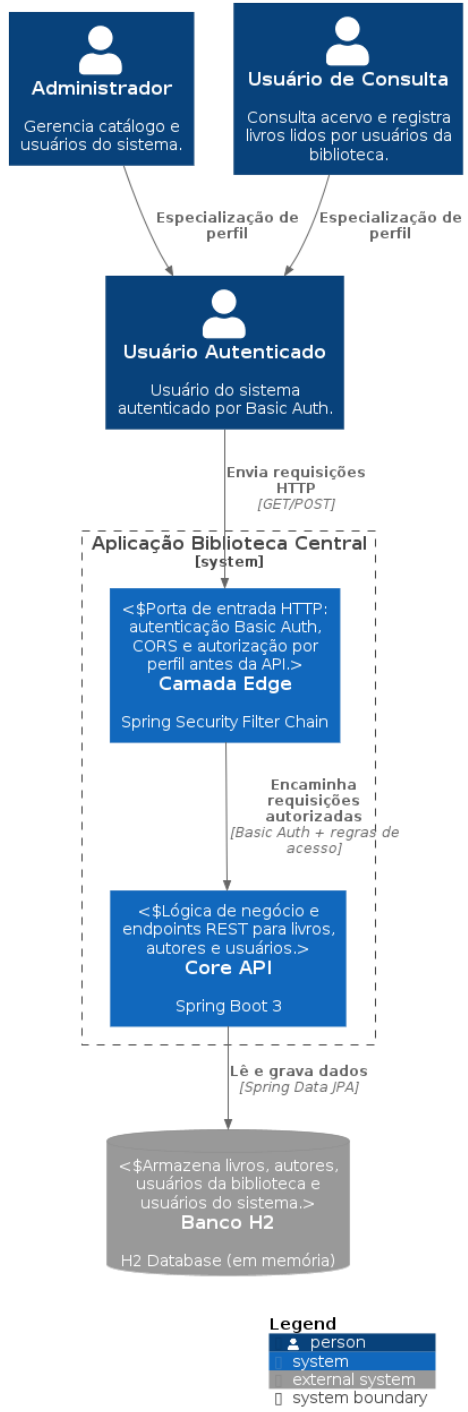
- 401 Unauthorized
- A requisição não possui credenciais válidas. O cliente precisa se autenticar.
- 403 Forbidden
- O usuário está autenticado, mas não possui autorização para executar a operação solicitada.
- No HTTP Basic, as credenciais são normalmente enviadas no cabeçalho Authorization em cada requisição protegida; o mecanismo não depende de um token de sessão.

Implementando autenticação básica com Spring Security

- Spring Security é um framework para autenticação, autorização e proteção contra ataques comuns.
- Ele atua na fronteira da aplicação, interceptando requisições antes que elas cheguem aos controllers.
- Neste exemplo, usaremos Spring Security para implementar HTTP Basic Authentication e controlar o acesso aos endpoints.
- O mesmo framework também oferece suporte a mecanismos de segurança mais sofisticados.

Arquitetura da solução

- A figura ao lado apresenta os principais componentes da solução da biblioteca usando HTTP Basic.
- O Spring Security atua na fronteira da aplicação: intercepta as requisições antes que elas cheguem aos controllers e aplica as regras de autenticação e autorização.
- Neste exemplo, os usuários do sistema são persistidos com JPA no banco H2.



1) O arquivo POM

- Estas são as principais dependências do projeto:
- Spring Web
- Spring Security
- JPA
- Validation
- H2 database
- JUnit

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

2) Arquivo application.properties

- As propriedades configuram o banco H2 usado no exemplo e habilitam recursos utilizados pela API.

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true

# Console H2 para testes (opcional)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Respostas de erro no padrão ProblemDetail
spring.mvc.problemdetails.enabled=true
```

3) A classe UsuarioSistema

- Os usuários da biblioteca são os que frequentam a biblioteca. Os usuários do sistema são os funcionários que operam o sistema.
- Para a autenticação, precisamos cadastrar os usuários do sistema, modelados pela classe ao lado.
- Os atributos id, username, senha e perfil armazenam as informações necessárias à autenticação e autorização.
- GenerationType.IDENTITY delega a geração do valor da chave primária à coluna identity do banco de dados.
- A senha armazenada não será a senha original: o serviço gravará seu hash usando BCrypt.

```java
@Entity
@Table(name = "usuarios_sistema")
public class UsuarioSistema {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String senha;

    @Column(nullable = false)
    private String perfil;

    protected UsuarioSistema() {}

    public UsuarioSistema(String username, String senha, String perfil){
        this.username = username;
        this.senha = senha;
        this.perfil = perfil;
    }

    public Long getId() { return id; }

    public String getUsername() { return username; }

    public String getSenha() { return senha; }

    public String getPerfil() { return perfil; }
}
```

4) O repositório e o DTO

- O repositório acrescenta ao CRUD padrão as consultas necessárias para localizar e verificar usuários do sistema.
- NovoUsuarioSistemaRequest é um DTO imutável, definido como record.
- As anotações de Bean Validation expressam as restrições de entrada antes que o Service execute as regras de negócio.

```
@Repository
public interface UsuarioSistemaRepository extends JpaRepository<UsuarioSistema, Long> {
    Optional<UsuarioSistema> findByUsername(String username);
    boolean existsByUsername(String username);
}
```

```
public record NovoUsuarioSistemaRequest(
    @NotBlank @Size(max = 100) String username,
    @NotBlank @Size(min = 8, max = 100) String senha,
    @Pattern(regexp = "(?i)ADMIN|CONSULTA",
             message = "deve ser ADMIN ou CONSULTA")
    String perfil) {

        public String perfilNormalizado() {
            if (perfil == null || perfil.isBlank()) return "CONSULTA";
            return "ADMIN".equals(perfil.trim().toUpperCase())?"ADMIN":"CONSULTA";
    }
}
```

5) A classe UsuarioSistemaService

- O Service recebe um DTO que já passou pela validação estrutural.
- A regra de negócio verifica se o username já existe e sinaliza a duplicidade por meio de uma exceção.
- PasswordEncoder gera o hash da senha com BCrypt antes da persistência. A senha original não é armazenada.
- Observe a separação de responsabilidades: DTO valida formato; Service aplica regras de negócio; Repository persiste.

```
@Service
public class UsuarioSistemaService {
    private final UsuarioSistemaRepository repository;
    private final PasswordEncoder passwordEncoder;

    @Autowired
    public UsuarioSistemaService(
            UsuarioSistemaRepository repository,
            PasswordEncoder passwordEncoder) {
        this.repository = repository;
        this.passwordEncoder = passwordEncoder;
    }

    public UsuarioSistema cadastraUsuarioSistema(NovoUsuarioSistemaRequest novoUsuario) {
        String username = novoUsuario.username().trim();
        String senha = novoUsuario.senha().trim();

        if (repository.existsByUsername(username)) {
            throw new RecursoDuplicadoException(
                    "Usuário já cadastrado");
        }

        var usuario = new UsuarioSistema(username,passwordEncoder.encode(senha),
                                         novoUsuario.perfilNormalizado());
        return repository.save(usuario);
    }
}
```

Como o Spring Security processa a requisição

Requisição HTTP

↓

SecurityFilterChain

↓

HTTP Basic extrai as credenciais

↓

UserDetailsService carrega o usuário

↓

PasswordEncoder verifica a senha

↓

Autorização por perfil → Controller

6) Configuração

- SecurityConfig reúne os beans usados pelo mecanismo de segurança.
- PasswordEncoder define co as senhas serão transforma em hash antes de serem armazenadas.
- UserDetailsService é a pont entre o Spring Security e nos repositório: recebe o username e carrega os dados do usuário.
- CommandLineRunner cria usuários iniciais apenas para facilitar os testes da aplicação.

```
@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    public UserDetailsService userDetailsService(
                                UsuarioSistemaRepository usuarioSistema
        return username -> usuarioSistemaRepository.findByUsername(use
                .map(usuario -> User.builder()
                        .username(usuario.getUsername())
                        .password(usuario.getSenha())
                        .roles(usuario.getPerfil())
                        .build())
                .orElseThrow(() -> new UsernameNotFoundException("Usuário não encontrado"));
    }

    dLineRunner inicializaUsuariosSistema(
                        UsuarioSistemaRepository usuarioSistemaRepository,
                        PasswordEncoder passwordEncoder) {
        gs -> {
        usuarioSistemaRepository.existsByUsername("admin")) {
        usuarioSistemaRepository.save(new UsuarioSistema("admin",
                                        passwordEncoder.encode("admin123"), "ADMIN"));
    }
    if (!usuarioSistemaRepository.existsByUsername("consulta")) {
        usuarioSistemaRepository.save(new UsuarioSistema("consulta",
                                        passwordEncoder.encode("consulta123"), "CONSULTA"));

    ilterChain securityFilterChain(HttpSecurity http) throws Exception {...}
```

Configuração do hash das senhas

Carregamento do usuário

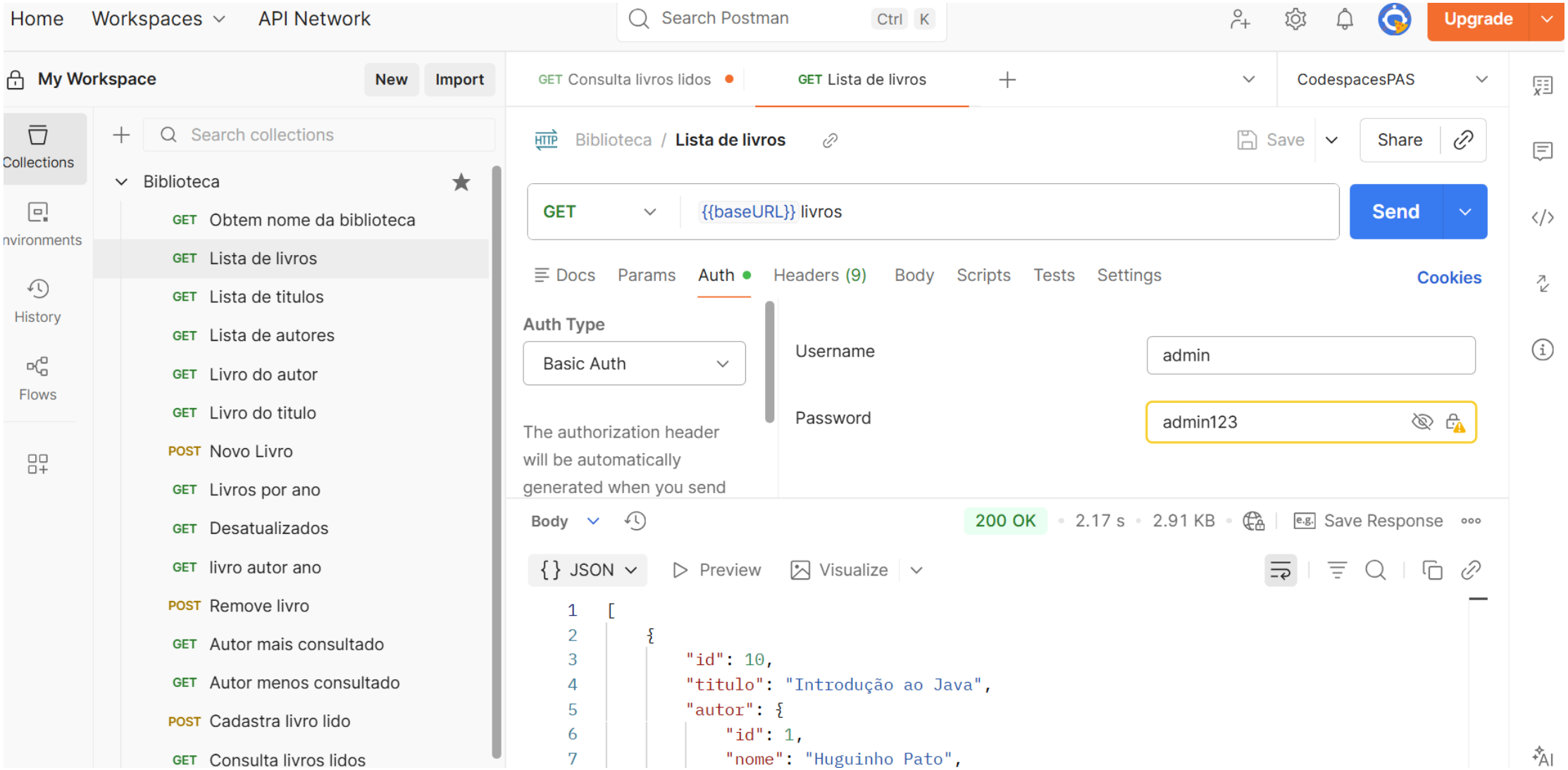
Usuários iniciais para testes

7)Configuração

- SecurityFilterChain define quais requisições podem chegar aos endpoints.
- As regras combinam método HTTP, rota e perfil do usuário.
- ADMIN pode alterar o acervo e cadastrar usuários; ADMIN e CONSULTA podem consultar dados e registrar livros lidos.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
        http
        .csrf(csrf -> csrf.disable())
        .cors(Customizer.withDefaults())
        .headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/h2-console/**").permitAll()
            .requestMatchers(HttpMethod.POST,"/livros", "/usuarios", "/usuarios-sistema")
                .hasRole("ADMIN")
            .requestMatchers(HttpMethod.DELETE, "/livros/**")
                .hasRole("ADMIN")
            .requestMatchers(HttpMethod.PUT,"/usuarios/*/livros-lidos/*")
                .hasAnyRole("ADMIN", "CONSULTA")
            .requestMatchers(HttpMethod.GET, "/**")
                .hasAnyRole("ADMIN", "CONSULTA")
            .anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

8) Testando a aplicação no Postman





Dinâmica

- Teste a API usando os usuários admin e consulta.

- Verifique: uma requisição sem credenciais → 401; uma operação autenticada, mas sem permissão → 403; um GET permitido aos dois perfis; e uma operação permitida apenas para ADMIN.

- Cadastre um novo usuário do sistema e confirme que ele consegue autenticar com a senha original, embora apenas o hash BCrypt seja armazenado no banco.